MyCodeSchool

Provocări — fără hinturi

10 exerciții fără indicații despre ce adnotare să folosești. Cerința e doar **funcțională**: ce trebuie să facă codul. Testele sunt sursa de adevăr — dacă trec testele, e bine.

Cum lucrezi: fiecare provocare are 1-3 clase scheleton și un test. Modifică **doar fișierele din `main/`** (clasele scheleton). NU modifica fișierele de test. Rulează:

```
# un singur test
./mvnw test -Dtest=C01GreeterTest
# toate provocările
./mvnw test -Dtest='com/example/fundamentals/provocari/**'
```

c01 — Greeter

`Greeter.greet("Alex")` trebuie să întoarcă `"Salut, Alex!"`. Clasa trebuie să poată fi injectată din contextul Spring.

c02 — LangWelcomer cu Translator

Există interfața `Translator` cu metoda `translate(name)`. Două implementări: `EngTranslator` (returnează `"Hello, <name>!"`) și `RomanianTranslator` (returnează `"Salut, <name>!"`). `LangWelcomer` primește un `Translator` și expune `welcome(name)` care delegă. Când nu specifici niciun qualifier la injecție, trebuie ales `EngTranslator`.

c03 — VatCalculator

`VatCalculator.calculate(amount)` întoarce `amount * rate`, unde `rate` e citit din proprietatea `app.vat.rate` (definită la test, ex: `0.19`). Pentru `amount=100` și `rate=0.19`, rezultatul = `19.0`.

c04 — RequestLogger

`RequestLogger` are `setRequestId(id)` și `getRequestId()`. Când cere de două ori prin `applicationContext.getBean(RequestLogger.class)`, trebuie să primești **două instanțe diferite** (nu același obiect).

c05 — CountryCache

`CountryCache.get(code)` întoarce numele țării: `"RO"` → `"Romania"`, `"DE"` → `"Germany"`, `"FR"` → `"France"`. Mapa internă trebuie populată **înainte ca prima cerere să sosească** — testul nu apelează nicio metodă de inițializare manuală.

c06 — MaintenanceService

`MaintenanceService.run()` întoarce `"DONE"`. Bean-ul trebuie să existe în context **DOAR** când e activ profilul `ops`. În alt context, `getBean(MaintenanceService.class)` aruncă `NoSuchBeanDefinitionException`.

c07 — Validators

Interfața `Validator` cu `validate(input)` (întoarce `boolean`). Două implementări: `EmailValidator` (cere `@`) și `PhoneValidator` (cere prefix `+`). `ValidationRunner.runAll(input)` întoarce `true` doar dacă **toate** implementările validează cu succes. Runner-ul nu cunoaște numărul de validatoare — primește lista direct prin constructor.

c08 — RandomConfig

O clasă de configurare publică 2 bean-uri: - `Random` configurat cu seed din proprietatea `app.random.seed` (ex: 42 → `nextInt(100) == 63`). - `DateTimeFormatter` cu pattern din `app.date.pattern` (ex: `"dd/MM/yyyy"` → `LocalDate.of(2026, 6, 4)` formatat = `"04/06/2026"`).

c09 — PaymentResolver

Interfața `PaymentStrategy` cu `pay(amount)` (întoarce un String). Două implementări: `CardPayment` (returnează `"CARD: <amount>"`) și `PaypalPayment` (returnează `"PAYPAL: <amount>"`). `PaymentResolver.resolve(key)` întoarce strategia potrivită: `"card"` → CardPayment, `"paypal"` →

PaypalPayment. Resolver-ul NU folosește if/switch pe tipul concret — primește direct o structură populată de Spring și caută în ea.

c10 — ConnectionPool

`ConnectionPool.getOpenConnections()` întoarce numărul de conexiuni deschise. Imediat după pornirea contextului Spring, valoarea trebuie să fie `10` (poll-ul s-a inițializat). Când se închide contextul, valoarea devine `0`.